Java NIO.2 — Medium-Level Practice Questions

20 Hands-on questions covering Path, Files, and Stream API

1. Path Information

Write a Java program that accepts a file path from the user and displays: File nameParent directoryRoot directory Absolute pathWhether the path is absolute or relative

2. Resolve a Path

Given the following:
Base directory: C:\College
Subdirectory: Students
File: marks.txt

Use Path.resolve() to construct the complete path C:\College\Students\marks.txt. Do not concatenate the paths using +.

3. Relative Path Using relativize()

Given:
C:\College\Students
C:\College\Teachers

Use Path.relativize() to find the relative path from Students to Teachers.

4. Create Directory and File

Write a program that: Creates a directory named CollegeData. Creates a file named students.txt inside it. Displays a suitable message if the file already exists. Use Files.createDirectories() and Files.createFile().

5. Copy a File

Write a program to copy students.txt to students_backup.txt. If students_backup.txt already exists, replace it. Use Files.copy() and StandardCopyOption.REPLACE_EXISTING.

6. Move and Rename a File

Create a file called oldname.txt and use Files.move() to rename it to newname.txt.

7. Delete If Exists

Write a program that asks the user for a file path and deletes the file only if it exists. Use Files.deleteIfExists(). Display an appropriate message in both cases.

8. File Properties

Write a program that accepts a file path and displays:
File Name:
File Size:
Is Regular File:
Is Directory:
Is Hidden:

Use appropriate methods from Files and Path.

9. Display Only Files

Given a directory path, use Files.list() and Stream API to display only the regular files directly inside that directory. Do not display subdirectory contents.

10. Display Files by Extension

Write a program using Files.list() that displays only .txt files from a given directory.

11. Count Files and Directories

Using Files.list(), find the number of files and number of directories present directly inside a given directory.

12. Find the Largest File

Use Files.list() and Stream API to find the largest regular file directly inside a directory. Display its file name and size in bytes.

13. Find .java Files Recursively

Given a directory, use Files.walk() to find all .java files, including those present inside subdirectories. Display their complete paths.

14. Count Files Recursively

Use Files.walk() to count the total number of regular files in a directory and all of its subdirectories.

15. Find Large Files

Write a program using Files.walk() that displays all files whose size is greater than 100 KB.

16. Find Files by Name

Ask the user to enter a file name, for example notes.txt. Use Files.walk() to search for that file in a directory and its subdirectories. Display its complete path if found.

17. Count Lines Containing a Word

A file data.txt contains multiple lines. Ask the user for a word and use Files.lines() to count how many lines contain that word.

18. Find Lines with Marks Above 80

Suppose students.txt contains:
Aman 75
Rahul 88
Simran 92
Neha 67
Karan 85

Use Files.lines() and Stream API to display students who scored more than 80.

19. Directory Report Generator

Write a program that accepts a directory path and generates the following report using Files.walk():
Total Files:
Total Directories:
Total .java Files:
Total .txt Files:

Use Stream API operations such as filter() and count().

20. File Backup System

Write a Java program that accepts a source directory and backup directory. The program should: Create the backup directory if it doesn't exist. Find all .txt files in the source directory and its subdirectories using Files.walk(). Copy the .txt files to the backup directory. Replace the file if it already exists. Display the number of files successfully copied. Concepts required: Path, Files.createDirectories(), Files.walk(), Stream API, filter(), Files.copy(), REPLACE_EXISTING, and count().